



Introduction to Q-Learning for Exotic Pricing

24/04/2019 //////////////

Olivier



From Bellman Equation to Policy Search



If we know the policy :

- Bellman equation
- Value Iteration
- Policy Evaluation

$$V^\pi(s) \leftarrow \sum_{s'} P(s'|s, \pi(s)) (R(s, \pi(s), s') + \gamma V^\pi(s'))$$

We look for the best policy :

$$\left(\begin{array}{l} V^*(s) \\ a^* = \pi(s) \end{array} \right) \leftarrow \max_a \sum_{s'} P(s'|s, a) (R(s, a, s') + \gamma V^*(s'))$$

From Policy Search to Q-Learning



We split the policy search in two steps:

$$\left(\begin{array}{l} V^*(s) \\ a^* = \pi(s) \end{array} \right) \leftarrow \max_a \sum_{s'} P(s'|s, a) (R(s, a, s') + \gamma V^*(s'))$$

$$Q^*(s, a) \leftarrow \sum_{s'} P(s'|s, a) \left(R(s, a, s') + \gamma \max_{a'} Q^*(s', a') \right)$$

$$V^*(s) = \max_{a'} Q^*(s, a')$$

From Value to Policy (deterministic π)



Value Iteration : non linear

$$V_k^*(s) \leftarrow \max_a \sum_{s'} P(s'|s, a) (R(s, a, s') + \gamma V_{k-1}^*(s'))$$

Policy Iteration for a given $\pi(s)$

$$V_k^*(s) \leftarrow \sum_{s'} P(s'|s, \pi(s)) (R(s, \pi(s), s') + \gamma V_{k-1}^*(s'))$$

At convergence we get V^π : the policy evaluation

$$V^\pi(s) \leftarrow \sum_{s'} P(s'|s, \pi(s)) (R(s, \pi(s), s') + \gamma V^\pi(s'))$$



Update Equation

$$V_k^\pi(s) \leftarrow \sum_{s'} \sum_a \pi(a|s) P(s'|s, a) (R(s, a, s') + \gamma V_{k-1}^\pi(s'))$$

Smooth out the policy,
improve the behavior of the optimization



Q-Value iteration (s and a are finite)

$$Q_k(s, a) \leftarrow \sum_{s'} P(s'|s, a) \left(R(s, a, s') + \gamma \max_{a'} Q_{k-1}(s', a') \right)$$

Tabular Q-learning with infinite state space s

- We sample on a till convergence before moving on k

$$Q_k(s, a) \leftarrow \alpha \left(R(s, a, s') + \gamma \max_{a'} Q_{k-1}(s', a') \right) + (1 - \alpha) Q_{k-1}(s, a)$$

3 conditions :

- You have to explore enough
- α has to be small enough
- but to decrease to quickly ..

Visit infinitely often every state

$$\sum_{t=0}^{\infty} \alpha_t(s, a) = \infty$$

$$\sum_{t=0}^{\infty} (\alpha_t)^2(s, a) < \infty$$

On Policy / Off Policy



Off Policy : Q learning

$$Q_k(s, a) \leftarrow \sum_{s'} P(s'|s, a) \left(R(s, a, s') + \gamma \max_{a'} Q_{k-1}(s', a') \right)$$

On Policy : SARSA

$$Q_k(s, a) \leftarrow \sum_{s'} P(s'|s, a) (R(s, a, s') + \gamma Q_{k-1}(s', a'))$$

Need a policy here !

Policy Improvement



Policy Iteration for a given $\pi_{k-1}(s)$

$$V_l^{\pi_{k-1}}(s) \leftarrow \sum_{s'} P(s'|s, \pi_{k-1}(s)) (R(s, \pi_{k-1}(s), s') + \gamma V_{l-1}^{\pi_{k-1}}(s'))$$

Policy Improvement

$$V_l^{\pi_k}(s) \leftarrow \arg \max_a \sum_{s'} P(s'|s, a) (R(s, a, s') + \gamma V_{l-1}^{\pi_{k-1}}(s'))$$

Problems and Solutions



- **Generalization coming from large number of state or actions.**
 - **Instability coming from the fleeing of the target.**
 - **Blowing coming temporal correlation due to online learning.**
-
- **Deep neural network as the Q value**
 - **Replay Buffer**
 - **Double parameter Set for the Q network with a regular update**

Approximate Q-learning



Algorithm:

Start with $Q_0(s, a)$ for all s, a .

Get initial state s

For $k = 1, 2, \dots$ till convergence

Sample action a , get next state s'

If s' is terminal:

$$\text{target} = R(s, a, s')$$

Sample new initial state s'

else:

$$\text{target} = R(s, a, s') + \gamma \max_{a'} Q_k(s', a')$$

$$\theta_{k+1} \leftarrow \theta_k - \alpha \nabla_{\theta} \mathbb{E}_{s' \sim P(s'|s,a)} [(Q_{\theta}(s, a) - \text{target}(s'))^2] \big|_{\theta=\theta_k}$$

$$s \leftarrow s'$$

Chasing a nonstationary target!

Updates are correlated within a trajectory!

Q-learning Improvement -1



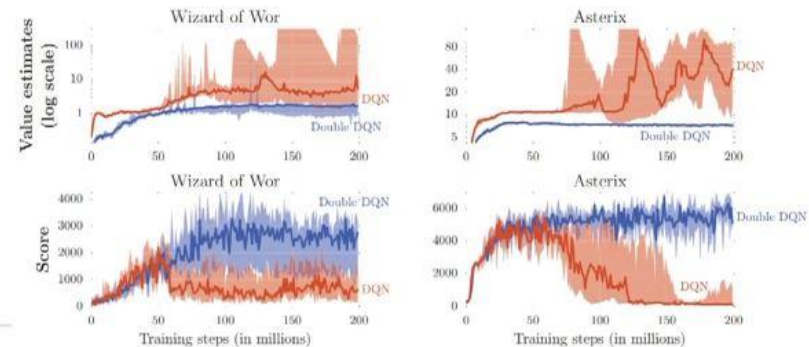
Double DQN

- There is an upward bias in $\max_a Q(s, a; \theta)$.
- DQN maintains two sets of weight θ and θ^- , so reduce bias by using:
 - θ for selecting the best action.
 - θ^- for evaluating the best action.

Double DQN loss:

$$L_i(\theta_i) = \mathbb{E}_{s,a,s',r \sim D} \left(r + \gamma Q(s', \arg \max_{a'} Q(s', a'; \theta); \theta_i^-) - Q(s, a; \theta_i) \right)^2$$

	no ops		human starts		
	DQN	DDQN	DQN	DDQN	DDQN (tuned)
Median	93%	115%	47%	88%	117%
Mean	241%	330%	122%	273%	475%



"Double Reinforcement Learning with Double Q-Learning", van Hasselt et al. (2016)

Q-learning Improvement -2



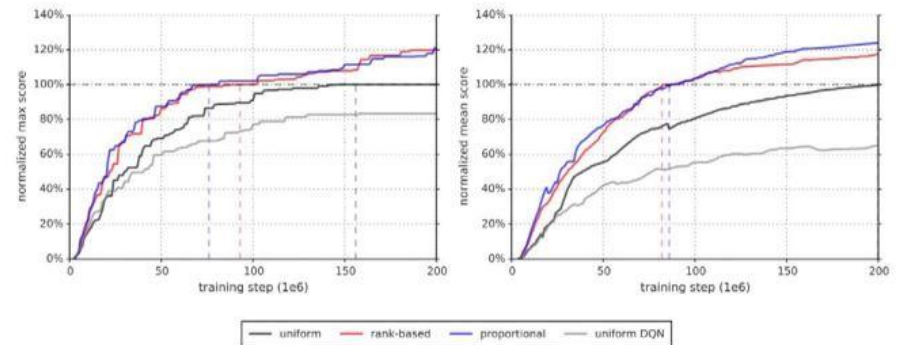
Prioritized Experience Replay

- Replaying all transitions with equal probability is highly suboptimal.
- Replay transitions in proportion to absolute Bellman error:

$$\left| r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right|$$

- Leads to much faster learning.

	DQN		Double DQN (tuned)		
	baseline	rank-based	baseline	rank-based	proportional
Median	48%	106%	111%	113%	128%
Mean	122%	355%	418%	454%	551%
> baseline	—	41	—	38	42
> human	15	25	30	33	33
# games	49	49	57	57	57



"Prioritized Experience Replay", Schaul et al. (2016)

Q-learning Improvement -3



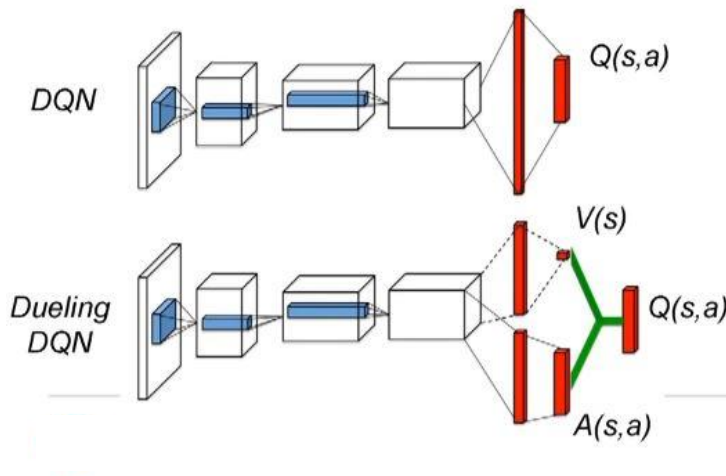
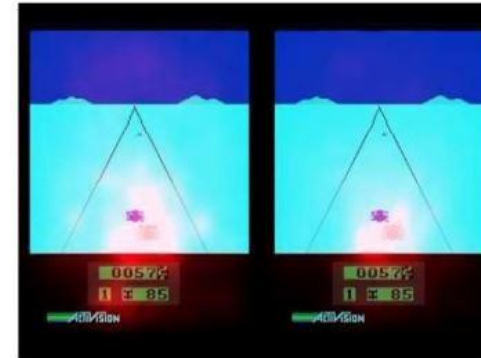
Dueling DQN

- Value-Advantage decomposition of Q:

$$Q^\pi(s, a) = V^\pi(s) + A^\pi(s, a)$$

- Dueling DQN (Wang et al., 2015):

$$Q(s, a) = V(s) + A(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a=1}^{|\mathcal{A}|} A(s, a)$$



Atari Results

	30 no-ops		Human Starts	
	Mean	Median	Mean	Median
Prior. Duel Clip	591.9%	172.1%	567.0%	115.3%
Prior. Single	434.6%	123.7%	386.7%	112.9%
Duel Clip	373.1%	151.5%	343.8%	117.1%
Single Clip	341.2%	132.6%	302.8%	114.1%
Single	307.3%	117.8%	332.9%	110.9%
Nature DQN	227.9%	79.1%	219.6%	68.5%

"Dueling Network Architectures for Deep Reinforcement Learning", Wang et al. (2016)

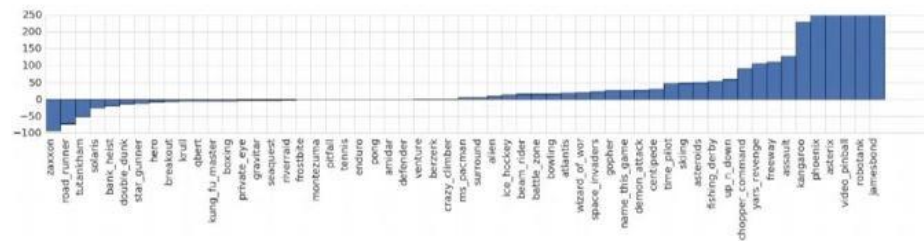
Q-learning Improvement -4



Noisy Nets for Exploration

- Add noise to network parameters for better exploration [Fortunato, Azar, Piot et al. (2017)].
- Standard linear layer: $y = wx + b$
- Noisy linear layer: $y \stackrel{\text{def}}{=} (\mu^w + \sigma^w \odot \varepsilon^w)x + \mu^b + \sigma^b \odot \varepsilon^b$
- ε^w and ε^b contain noise.
- σ^w and σ^b are learned parameters that determine the amount of noise.

	Baseline		NoisyNet	
	Mean	Median	Mean	Median
DQN	213	47	1210	89
A3C	418	93	1112	121
Dueling	2102	126	1908	154



"Noisy Nets for Exploration", Fortunato, Azar, Piot et al. (2017)
Also see "Parameter Space Noise for Exploration", Plappert et al. (2017)

Policy Optimization



Comparaison Policy Optim./ Dyn.Prog.



	Policy Optimization	Dynamic Programming
Conceptually	Optimize what you care about	Indirect, exploit the structure, self-consistency
Empirically	More compatible with rich architectures : recurrence,...	More compatible with exploration/exploitation and off policy

Likelihood Ratio Gradient



$$U(\theta) = \sum_{\tau} P(\tau; \theta) R(\tau)$$

Taking the gradient w.r.t. θ gives

$$\begin{aligned} \nabla_{\theta} U(\theta) &= \nabla_{\theta} \sum_{\tau} P(\tau; \theta) R(\tau) \\ &= \sum_{\tau} \nabla_{\theta} P(\tau; \theta) R(\tau) \\ &= \sum_{\tau} \frac{P(\tau; \theta)}{P(\tau; \theta)} \nabla_{\theta} P(\tau; \theta) R(\tau) \\ &= \sum_{\tau} P(\tau; \theta) \frac{\nabla_{\theta} P(\tau; \theta)}{P(\tau; \theta)} R(\tau) \\ &= \sum_{\tau} P(\tau; \theta) \nabla_{\theta} \log P(\tau; \theta) R(\tau) \end{aligned}$$

Approximate with the empirical estimate for m sample paths under policy π_{θ} :

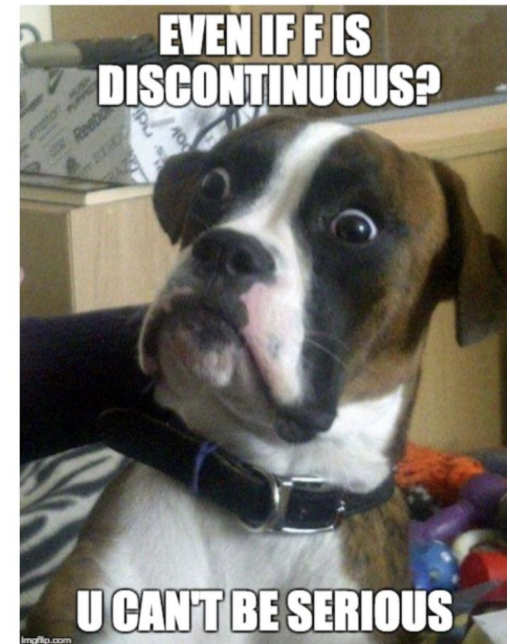
$$\nabla_{\theta} U(\theta) \approx \hat{g} = \frac{1}{m} \sum_{i=1}^m \nabla_{\theta} \log P(\tau^{(i)}; \theta) R(\tau^{(i)})$$

Likelihood Ratio Gradient : Validity



$$\nabla U(\theta) \approx \hat{g} = \frac{1}{m} \sum_{i=1}^m \nabla_{\theta} \log P(\tau^{(i)}; \theta) R(\tau^{(i)})$$

- Valid even when
 - R is discontinuous and/or unknown
 - Sample space (of paths) is a discrete set



From Importance Sampling



$$U(\theta) = \mathbb{E}_{\tau \sim \theta_{\text{old}}} \left[\frac{P(\tau|\theta)}{P(\tau|\theta_{\text{old}})} R(\tau) \right]$$

$$\nabla_{\theta} U(\theta) = \mathbb{E}_{\tau \sim \theta_{\text{old}}} \left[\frac{\nabla_{\theta} P(\tau|\theta)}{P(\tau|\theta_{\text{old}})} R(\tau) \right]$$

$$\nabla_{\theta} U(\theta)|_{\theta=\theta_{\text{old}}} = \mathbb{E}_{\tau \sim \theta_{\text{old}}} \left[\frac{\nabla_{\theta} P(\tau|\theta)|_{\theta_{\text{old}}}}{P(\tau|\theta_{\text{old}})} R(\tau) \right]$$

$$= \mathbb{E}_{\tau \sim \theta_{\text{old}}} \left[\nabla_{\theta} \log P(\tau|\theta)|_{\theta_{\text{old}}} R(\tau) \right]$$

Suggests we can also look at more than just gradient!

E.g., can use importance sampled objective as “surrogate loss” (locally)

Decomposition of Paths in States & Actions



$$\begin{aligned}\nabla_{\theta} \log P(\tau^{(i)}; \theta) &= \nabla_{\theta} \log \left[\prod_{t=0}^H \underbrace{P(s_{t+1}^{(i)} | s_t^{(i)}, u_t^{(i)})}_{\text{dynamics model}} \cdot \underbrace{\pi_{\theta}(u_t^{(i)} | s_t^{(i)})}_{\text{policy}} \right] \\&= \nabla_{\theta} \left[\sum_{t=0}^H \log P(s_{t+1}^{(i)} | s_t^{(i)}, u_t^{(i)}) + \sum_{t=0}^H \log \pi_{\theta}(u_t^{(i)} | s_t^{(i)}) \right] \\&= \nabla_{\theta} \sum_{t=0}^H \log \pi_{\theta}(u_t^{(i)} | s_t^{(i)}) \\&= \sum_{t=0}^H \underbrace{\nabla_{\theta} \log \pi_{\theta}(u_t^{(i)} | s_t^{(i)})}_{\text{no dynamics model required!!}}\end{aligned}$$

Likelihood Ratio Gradient Estimate



- **As formulated thus far : unbiased but very noisy**
- **Fixes that lead to real world practicality**
 - **Baseline**
 - **Temporal structures**
 - **KL-divergence trust region / natural gradient**



- We will use a softmax policy as a running example
- Weight actions using linear combination of features $\phi(s, a)^\top \theta$
- Probability of action is proportional to exponentiated weight

$$\pi_\theta(s, a) \propto e^{\phi(s, a)^\top \theta}$$

- The score function is

$$\nabla_\theta \log \pi_\theta(s, a) = \phi(s, a) - \mathbb{E}_{\pi_\theta} [\phi(s, \cdot)]$$



- In continuous action spaces, a Gaussian policy is natural
- Mean is a linear combination of state features $\mu(s) = \phi(s)^\top \theta$
- Variance may be fixed σ^2 , or can also be parametrised
- Policy is Gaussian, $a \sim \mathcal{N}(\mu(s), \sigma^2)$
- The score function is

$$\nabla_{\theta} \log \pi_{\theta}(s, a) = \frac{(a - \mu(s))\phi(s)}{\sigma^2}$$

Reducing Variance using Critik



- Monte-Carlo policy gradient still has high variance
- We use a **critic** to estimate the action-value function,

$$Q_w(s, a) \approx Q^{\pi_\theta}(s, a)$$

- Actor-critic algorithms maintain *two* sets of parameters
 - Critic** Updates action-value function parameters w
 - Actor** Updates policy parameters θ , in direction suggested by critic
- Actor-critic algorithms follow an *approximate* policy gradient

$$\nabla_\theta J(\theta) \approx \mathbb{E}_{\pi_\theta} [\nabla_\theta \log \pi_\theta(s, a) Q_w(s, a)]$$

$$\Delta\theta = \alpha \nabla_\theta \log \pi_\theta(s, a) Q_w(s, a)$$

Actor Critik Algorithm



- Simple actor-critic algorithm based on action-value critic
- Using linear value fn approx. $Q_w(s, a) = \phi(s, a)^\top w$
 - Critic Updates w by linear TD(0)
 - Actor Updates θ by policy gradient

function QAC

 Initialise s, θ

 Sample $a \sim \pi_\theta$

for each step **do**

 Sample reward $r = \mathcal{R}_s^a$; sample transition $s' \sim \mathcal{P}_{s,\cdot}^a$.

 Sample action $a' \sim \pi_\theta(s', a')$

$\delta = r + \gamma Q_w(s', a') - Q_w(s, a)$

$\theta = \theta + \alpha \nabla_\theta \log \pi_\theta(s, a) Q_w(s, a)$

$w \leftarrow w + \beta \delta \phi(s, a)$

$a \leftarrow a', s \leftarrow s'$

end for

end function

Compatible Function Approximation

Theorem

If the following two conditions are satisfied:

- 1** *Value function approximator is **compatible** to the policy*

$$\nabla_w Q_w(s, a) = \nabla_\theta \log \pi_\theta(s, a)$$

- 2** *Value function parameters w minimise the mean-squared error*

$$\varepsilon = \mathbb{E}_{\pi_\theta} [(Q^{\pi_\theta}(s, a) - Q_w(s, a))^2]$$

Then the policy gradient is exact,

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta} [\nabla_\theta \log \pi_\theta(s, a) Q_w(s, a)]$$

Reducing Variance using a Baseline

- We subtract a baseline function $B(s)$ from the policy gradient
- This can reduce variance, without changing expectation

$$\begin{aligned}\mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(s, a) B(s)] &= \sum_{s \in \mathcal{S}} d^{\pi_{\theta}}(s) \sum_a \nabla_{\theta} \pi_{\theta}(s, a) B(s) \\ &= \sum_{s \in \mathcal{S}} d^{\pi_{\theta}} B(s) \nabla_{\theta} \sum_{a \in \mathcal{A}} \pi_{\theta}(s, a) \\ &= 0\end{aligned}$$

- A good baseline is the state value function $B(s) = V^{\pi_{\theta}}(s)$
- So we can rewrite the policy gradient using the **advantage function** $A^{\pi_{\theta}}(s, a)$

$$A^{\pi_{\theta}}(s, a) = Q^{\pi_{\theta}}(s, a) - V^{\pi_{\theta}}(s)$$

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(s, a) A^{\pi_{\theta}}(s, a)]$$

Natural Policy Gradient

- The **natural policy gradient** is parametrisation independent
- It finds ascent direction that is closest to vanilla gradient, when changing policy by a small, fixed amount

$$\nabla_{\theta}^{\text{nat}} \pi_{\theta}(s, a) = G_{\theta}^{-1} \nabla_{\theta} \pi_{\theta}(s, a)$$

- where G_{θ} is the Fisher information matrix

$$G_{\theta} = \mathbb{E}_{\pi_{\theta}} \left[\nabla_{\theta} \log \pi_{\theta}(s, a) \nabla_{\theta} \log \pi_{\theta}(s, a)^T \right]$$

- So the natural policy gradient simplifies, $\nabla_w A_w(s, a) = \nabla_{\theta} \log \pi_{\theta}(s, a)$

$$\begin{aligned} \nabla_{\theta} J(\theta) &= \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(s, a) A^{\pi_{\theta}}(s, a)] \\ &= \mathbb{E}_{\pi_{\theta}} \left[\nabla_{\theta} \log \pi_{\theta}(s, a) \nabla_{\theta} \log \pi_{\theta}(s, a)^T w \right] \\ &= G_{\theta} w \end{aligned}$$

$$\nabla_{\theta}^{\text{nat}} J(\theta) = w$$

Update actor in direction of the critic